

Antarctic Vision: Detecting Penguins with YOLOv5

Benjamin Philipenko

*Dept. of Computer Engineering University of
Victoria*

Victoria, Canada

Bentley Barth

*Dept. of Electrical Engineering University of
Victoria*

Victoria, Canada.

Abstract - By utilizing a single-stage detector network known as YOLOv5 (You Only Look Once), users can train and detect specific objects of various sizes with a certain level of confidence based on the accuracy and precision of the model [1]. Implementation of the YOLOv5 involves gathering images to build a dataset, training the YOLOv5 model to recognise desired objects within the dataset, and assessing the models confidence.

Keywords - Single-Stage Detector Network, YOLOv5, dataset, level of confidence

I. INTRODUCTION

The main purpose of this project was to develop a computer vision model capable of identifying penguins in an antarctic themed picture or video, regardless of the size or orientation of the penguin. Utilizing the YOLOv5 (You Only Look Once) object detection model, our objective was to generate output images with bounding boxes that precisely outline all identified features [1]. Additionally, our aim was to also perform feature segmentation on the output images. This process involves identifying and highlighting individual objects within the images at the pixel level. The goal of feature segmentation is to assign a class label to each pixel, as well as distinguish between different instances of the same class. This is a complex task that poses considerable challenges, especially when dealing with multiple features in a single image [2]. Before the project could commence, it was necessary to conduct comprehensive research on the current methodologies for object detection.

A. Overview of Literature Review for Insights

To establish a foundation for implementing object detection, we reviewed various research papers and

articles that explored alternate object detection methods. A leaf disease detection app known as Flask, provided a great source of inspiration [3]. Flask uses a deep learning model that can predict what type of leaf disease a plant may have based on images uploaded by users. This in turn helps users diagnose the disease, enabling them to identify proper treatment methods. After reviewing this paper, it became apparent that object detection will be crucial in the future. Recognizing that a simple application like Flask can provide valuable assistance to people further fueled the desire to build an object detection model. Although Flask is used for detecting minor issues like leaf diseases, other companies like HealthCure have designed and trained models to detect significant medical conditions [4]. Some medical conditions that the model can detect include:

- Covid-19 Detection.
- Brain Tumor Detection.
- Alzheimer Detection.
- Diabetes Detection.
- Pneumonia Detection.
- Heart Disease Detection.

HealthCure utilizes a Convolutional Neural Network deep learning algorithm that can take an “input image, assign importance (learnable weights and biases) to various aspects/objects in the image, and be able to differentiate one from the other” [4].

B. Configuring the Environment

Once the research phase was completed, the project could commence. Google CoLab was selected as the primary environment to train the YOLOv5 model and undertake various coding tasks. To effectively utilize YOLOv5, users must first clone the YOLOv5 GitHub Repository into the Google CoLab environment. The cloning procedure installs all the required dependencies and modules to ensure the environment is prepared to use the full capabilities of the YOLOv5.

C. Method 1: Pre-trained Model

To further gain an understanding of the YOLOv5 single-stage detector network, two methods were explored. The first method involved utilizing the pre-trained models and built-in datasets. These “built-in datasets” contain 80 predefined classes, ranging from common objects such as birds, bottles, cats, persons, vehicles, etc. This provided a simple “plug and play” method, which delivered high accuracy, precision, and confidence in performing object detection and instance segmentation. By experimenting with the capabilities of the pre-trained models, we were able to extend its object detection proficiency to videos. This highlighted an innovative aspect of the project, as the ability to apply object detection to video content has significant real-world applications, especially as the need for automated image analysis to aid environmental conservation efforts increases. A short video demonstration was created to showcase potential applications, such as environmental monitoring and wildlife conservation efforts.

Antarctic Penguin Detection: YOLOv5 Object Detection Model in Action

Although the pre-trained models were able to produce exceptionally accurate results, we were confined to the default classes of the predefined datasets. This highlighted the need for a custom model trained on antarctic image datasets, which could satisfy our project specific requirements. The absence of a dedicated class for penguins presented a challenge, as the pre-trained models classified penguin object instances as “birds.” This stressed the importance of training a custom model capable of detecting objects outside of the predefined classes, which extended the scope of our work beyond the limits of the pre-trained datasets.

D. Method 2: Custom Dataset Model

YOLOv5 offers the flexibility for users to train a model using a custom dataset, allowing them to extend beyond the limits of the pre-trained models.

With the use of a platform known as Roboflow, our team developed a custom dataset to train a custom object detection model. Roboflow offers robust tools that help users preprocess, annotate, and define classes for selected images. Roboflow implements many preprocessing features that ensure datasets are standardized before model training. This standardization is facilitated through tools such as auto-orienting images, converting to grayscale, adjusting contrast levels, etc.

Another feature of Roboflow that enhances dataset quality and model generalization are the augmentation options available. These tools perform different image augmentations such as flipping, rotating, cropping, blurring, and adding random noise to diversify the dataset, which ultimately produces a more robust model capable of accurate object detections.

To create the custom dataset in Roboflow, a database of 800 pre-annotated images was imported from Google’s Open Images Dataset V7 platform via Google’s API [5]. It was crucial to review the annotated images within the uploaded dataset to ensure that the dataset accurately represented penguins. This data was used to implement the second method in our project experimentation, which successfully overcame the challenge of mislabelling penguins as “birds” within the predefined classes.

II. EXPERIMENTS AND ANALYSIS

A. Method 1: Pre-Trained Model

By utilizing the pre-trained models and built-in datasets, implementing object detection and segmentation proved to be a straightforward and efficient process. The initial setup involved creating a virtual environment, which ensures consistency across different user environments. The YOLOv5 repository was then cloned into the Google Colab environment, which provided access to the YOLOv5 model and its associated scripts for object detection and segmentation. All the dependencies and modules required to run the YOLOv5 vision AI are also imported during the cloning process.

To evaluate the pre-trained model performance, a simple test bench was created on a public google drive of 3 *.jpeg* files and 1 *.mov* file, which all contained images of penguin scenes. These files were then imported from the google drive and used as samples for initial input during object detection and segmentation testing. The *detect.py* script was run on sample images to perform object detection and the *segment/predict.py* script was used to perform instance segmentation. The results of the pre-trained model object detection and segmentation testing can be seen in Figure 1 and Figure 2, respectively.

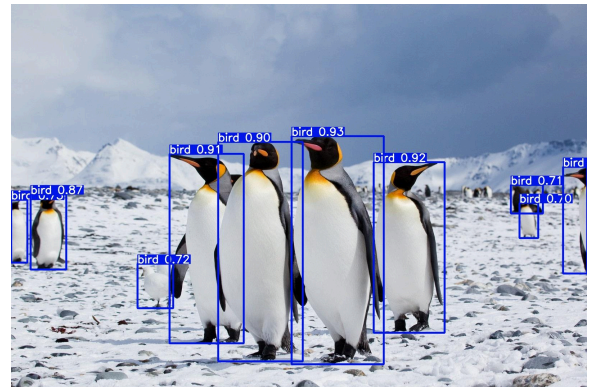


Figure 1: Pre-trained Model, Result: Bird

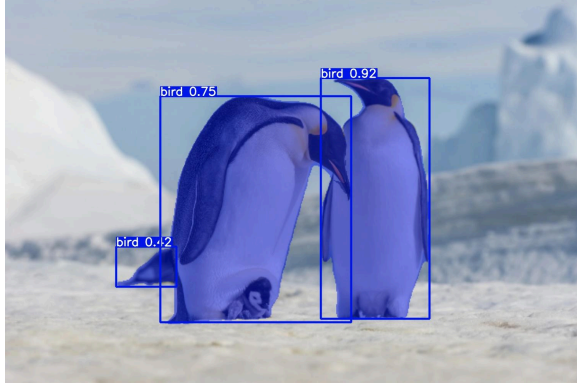


Figure 2: Pre-trained Model, Segmentation

The pre-trained models were able to deliver impressive accuracy, with all detected features achieving accuracy ratings greater than 70%, despite being confined to the default classes of the predefined datasets. As previously stated, the penguins were classified with a generic label, “bird,” which motivated the development of an alternative method. In real-world wildlife monitoring applications, generic labels do not suffice, as models must be capable of identifying breed, age, gender, and other characteristics to truly impact wildlife conservation applications.

B. Method 2: Custom Dataset Model

The second method involved developing a model built to specifically target penguins and Antarctic landscape features. Training a model using a custom dataset requires a significant amount of time and computational power, which was a crucial consideration during development.

To begin, three distinct datasets were compiled and imported into Roboflow to be prepared for machine learning tasks. As previously stated, the Roboflow platform was implemented for formatting, annotating and pre-processing our images, which ensured consistency and enhanced the model’s ability to perform effectively on unseen images. The first dataset contained 1600 pre-annotated and augmented images of penguins in antarctic settings, which was a custom dataset developed by our team. The other two datasets contained rock and iceberg instance segmentation data, generated respectively by Oregon State University [6] and an independent research team in France [7], all processed via Roboflow.

To initiate the training, the Roboflow Python package was installed, and linked to the YOLOv5 model within the Google CoLab environment via the Application Programming Interface (API) key from Roboflow. Each dataset was then retrieved using unique project and version identifiers, and imported into the CoLab environment to commence the empirical experimental process of training the YOLOv5 model.

To configure the training execution, there are adjustable training parameters that greatly affect the

overall performance of the model, which make this the “empirical experimental process” of the project. These adjustable parameters include image size, batch size, and the number of epochs.

The image size is defined by a numerical value, N , that sets the dimensions of the input image to $N \times N$. This can greatly alter the training duration of the model; if the image size is smaller, the model processes fewer pixels per image, which accelerates the training. However, depending on the specific dataset and task, a reduced image size can potentially lead to a decrease in the models overall performance and accuracy [8]. The batch size is defined by a numerical value that defines the number of images processed in each iteration of the training process. Batch size also greatly affects the time required to train the model. A smaller batch size corresponds to more iterations required to process the entire dataset compared to larger batch sizes. The batch size affects the generalization performance and computational efficiency, and often requires empirical testing to determine the optimal size [9]. The number of epochs represents the number of iterations the entire dataset will be passed through the model for training. This parameter directly influences the training time of the model. For example, if the number of epochs is set to 1, then only a single iteration of processing the dataset will be executed during training. Typically, it is beneficial to maximize the number of epochs, which allows the model to encounter the dataset multiple times, and learn more complex patterns. It should be noted that overly large epoch sizes can lead to overfitting [10], wherein the model overly adapts to the training data, which compromises its ability to perform on unseen data.

In the first attempt to obtain suitable parameter weights, the image size was set to 740, batch size to 16, and number of epochs to 150. However, this resulted in CoLab timing out due to limited computing power, when implementing the T4 GPU hardware accelerator in runtime settings. In the next attempt, the parameters were significantly reduced: the image size was set to 416, batch size to 16, and number of epochs to 1. This adjustment greatly reduced the training duration. However, the resulting model demonstrated poor performance in accuracy, and precision, and in turn had a low level of confidence. The impact of these reduced parameters on the model’s efficacy is illustrated in Figure 3.

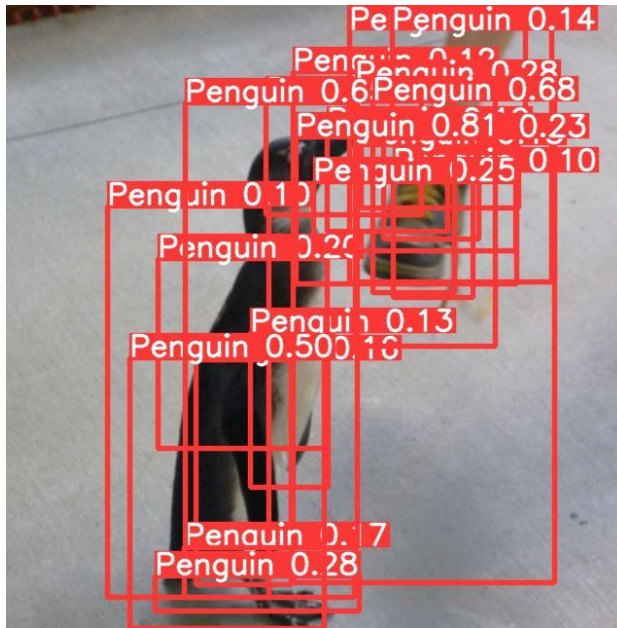


Figure 3: Custom, Extreme Reduction in Training Parameters

As can be seen in Figure 3, the under-trained model incorrectly identifies multiple instances of penguins despite only a single penguin instance present in the image. To correct these results, the number of epochs was increased to 10, while the image size and batch size were kept consistent with the previous training attempt. This extended the total training duration to 4 hours and 31 minutes. The resulting model demonstrated much higher levels of accuracy, precision, and confidence compared to the previous iteration. The effectiveness of these adjustments can be seen in the output images observed in Figures 4 and 5.



Figure 4: Custom, Group of Penguins and Elephant Seal

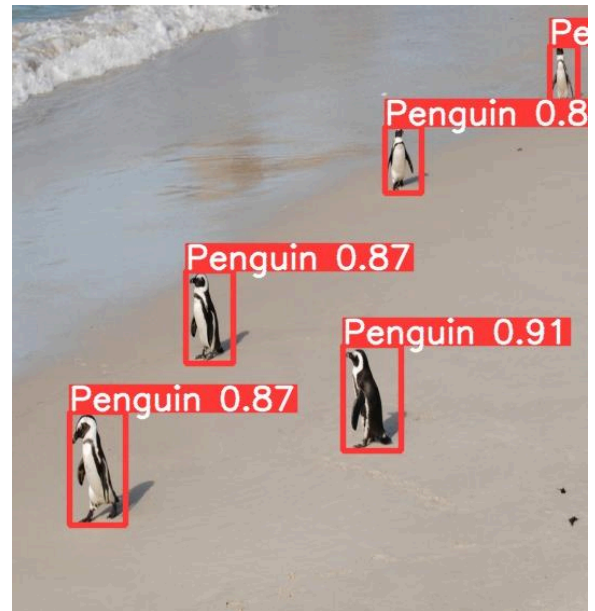


Figure 5: Custom, Group of Penguins on a Beach

To accurately assess the precision of the newly trained model, a Precision-Confidence Curve was generated using TensorBoard. Precision is the proportion of true positive predictions correctly identified among all positive predictions made by the model [11]. Confidence is a metric assigned by the model to represent its confidence level in its predictions [11]. Figure 6 showcases the Precision-Confidence Curve generated, where the x-axis represents confidence thresholds, and the y-axis represents precision. As the confidence threshold increases along the x-axis, the precision of the model's predictions on the y-axis also increases. For example, at a confidence threshold of 0.940, the precision for accurately detecting penguins in an image reaches 1.0. This indicates that all positive predictions made by the model at this threshold are correct [11].

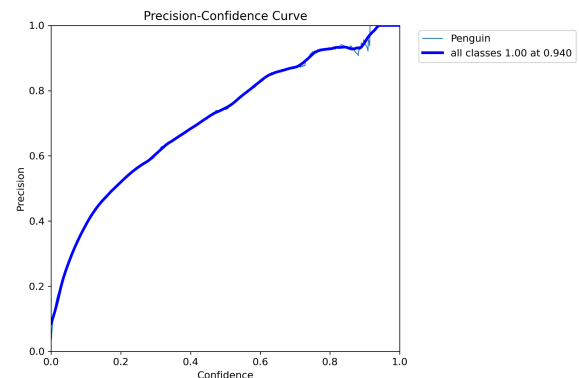


Figure 6: Precision-Confidence Curve

To assess the recall and confidence levels of the newly trained model, a Recall-Confidence Curve was generated, also using TensorBoard. Recall is a measure of the model's ability to correctly identify all relevant instances of penguins within images. Figure

7 illustrates that as confidence levels increase, the recall rate decreases. This indicates that the model becomes more selective in its predictions [11].

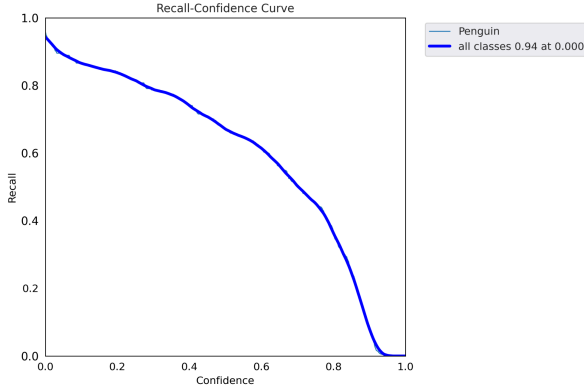


Figure 7: Recall-Confidence Curve

To further analyze the model's performance, a Precision-Recall Curve was generated. The area under the curve in Figure 8 represents the overall performance of the model [11]. In this training iteration, the penguin precision-recall curve indicates a precision of 0.753. This indicates the model is correct approximately 75.3% of the time.

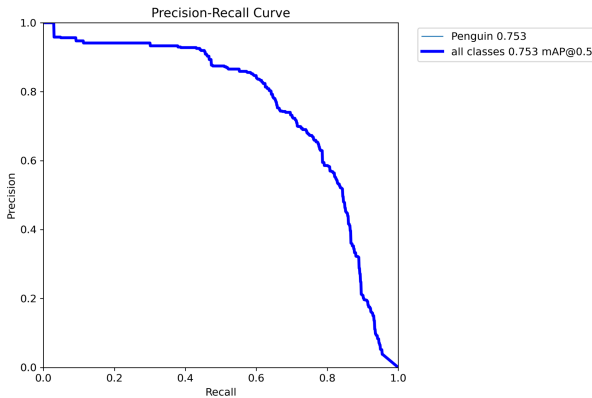


Figure 8: Precision-Recall Curve

C. Comparison of Object Detection Methods: Pre-Trained vs. Custom Dataset Approach

By completing both methods in our experimentation, we were able to exemplify the tradeoff between computational resources and model performance within object detection and instance segmentation. In our project, we implemented and compared the performance of the YOLOv5 pretrained models and our custom model developed for identifying penguins and Antarctic landscape features. These results highlight the optimization challenges created by limited computational resources and real-life application constraints. In Method 1, we implemented the pretrained models and built-in datasets, which allowed us to complete rapid prototyping and testing, while also achieving impressive accuracy in feature identification. However, being confined to the built-in dataset

classes demonstrated the limitations of the pre-trained models, as the model provided penguins with the generic "bird" label. Real-world wildlife applications require more detailed attribute and class identification, which necessitated an alternative method targeted specifically to our project objectives.

To mitigate the challenges we encountered using the pretrained model, we developed a custom dataset model, designed to specifically target penguins and Antarctic landscape features. This process required a large amount of computational resources and time, which demonstrated the empirical experimental process involved in training a model from scratch.

Our experiments highlighted the importance of computational power in accurately modeling cost analysis that must be assessed to satisfy application requirements and constraints. The pre-trained model method offered efficiency but was limited in its adaptability, while the custom dataset model required significantly more computational resources when attempting to optimize training parameters such as image size, batch size, and number of epochs.

In real-life applications such as Antarctic wildlife tracking, determining the optimal balance between accuracy and resource constraints is critical, as computational resources and data transmission are limited in remote settings. This optimization challenge manifested itself in our project as we encountered difficulties working within the limitations of the Google CoLab environment, which restricted us to the default T4 GPU within our runtime [12]. The performance of our custom dataset model could have been improved if more powerful hardware was accessible, such as the V100 GPU, which provides much greater processing power [13].

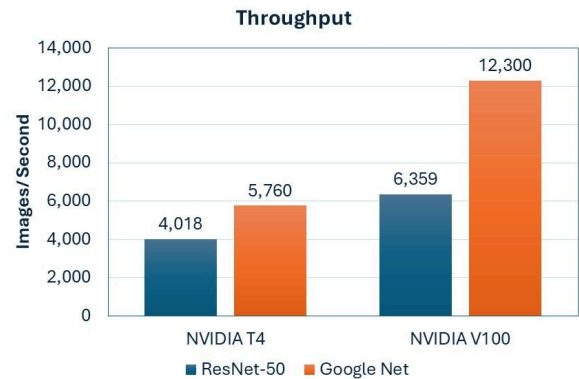


Figure 9: Comparison of Throughput between T4 and V100 GPUs

Another example demonstrating the trade-off between computational resources and model accuracy occurred during the development of the demonstration video produced while experimenting with pre-trained models. When performing object detection on a video file, various input parameters can be adjusted that affect the computational requirements, video quality, and accuracy of the output. Parameters such as frames per second and

step size directly influence the result of the corresponding output.

Object detection is performed on a video file by segmenting the video into a series of individual images frame by frame. Object detection is then performed on each frame, creating bounding boxes that precisely outline all identified features within the image. Depending on the computational resources available, adjusting the step size to a very small value and increasing the frames per second will produce a smoother and more accurate video output. However, this can significantly increase processing time, which becomes a critical consideration in real-life applications where real-time processing must be facilitated, such as in animal population surveys or habitat monitoring applications.

III. FUTURE WORK AND CONCLUSION

A. Conclusion

In this project, our objective was to develop a computer vision model capable of accurately identifying penguins in Antarctic-themed images or videos using the YOLOv5 object detection model. By implementing both the pretrained models and developing a model trained via a custom dataset, we observed an interesting optimization problem: balancing the computational resources available and model performance application requirements and constraints. Through our project experimentation, we demonstrated the capabilities and effectiveness of both methods, and illustrated the challenges encountered in real-world applications with limited computational resources available.

The pretrained models provided quick and efficient methods for performing object detection, achieving outstanding accuracy in identifying features despite being confined to the predefined classes of the built-in datasets. However, our project required a model capable of not only identifying birds but also specifically recognizing penguins, with the potential to extend its capabilities to identify even more subtle attributes, such as breed or gender. Developing a model trained on a custom dataset allowed us to mitigate this limitation but required significant computational resources and processing time to optimize training parameters.

Our project highlights the importance of balancing computational power with model accuracy, especially in real-life applications such as remote Antarctic wildlife observation applications, where resources are limited. While the pretrained models offer efficiency, they lack adaptability for specialized tasks. Custom dataset models offer flexibility but require substantial computational resources for training.

B. Future Work

Our project was successfully able to meet all objectives, however, there is potential for future improvements and extending its applications.

By continuing to refine the optimal training parameters such as image size, batch size, and number of epochs, the performance of the custom dataset model can be improved. Exploring solutions for optimal parameters may possibly lead to better accuracy and precision in penguin detection.

Additionally, by further refining the model to distinguish different species of penguins, as well as identifying attributes such as breed or gender, its utility in conservation efforts will greatly increase. By incorporating a range of measurements and defining features such as size, coloration, plumage patterns, bill shape, flipper size, and distinctive markings, the model can enhance its capability to determine penguin breeds and characteristics [14]. This expanded dataset will enable the model to refine its detection of penguins, encompassing both species identification and the recognition of subtle characteristics within each species.

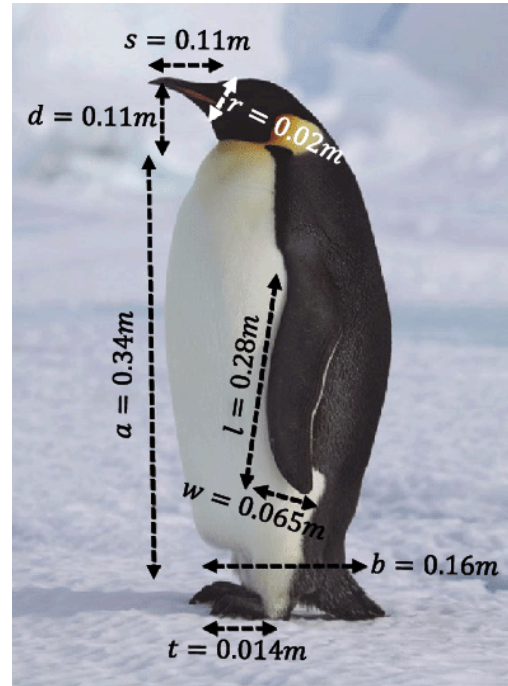


Figure 10: Measured Model of Emperor Penguin

Another option to explore is extending the project's object detection capabilities to other wildlife species and Antarctic environmental features. These expansions of the model's capabilities would greatly improve its impact in remote environments where detailed data on penguin populations is crucial for conservation management.

One other potential improvement to consider is integrating additional data sources into the custom model's training dataset. Incorporating additional datasets and image media, such as satellite imagery

or environmental data, would extend our model's capabilities for penguin detection and habitat monitoring. Figure 11 depicts satellite images of "a newly discovered emperor penguin colony, using satellite images of one the most remote and inaccessible regions of Antarctica," which was discovered in January 2023 [15].

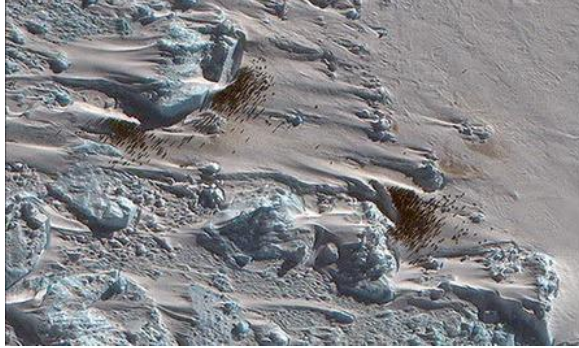


Figure 11: Emperor Penguin Colony Discovered Using Satellite Imaging

The emperor penguin colony depicted in Figure 11 is one of 66 known emperor penguin colonies, housing approximately 500 birds. These colonies are scattered along the Antarctic coastline, with half of these colonies being discovered through observations from space satellites. The increasing impact of climate change threatens these colonies, primarily due to the rapid melting of sea ice. These colonies serve as crucial habitats for the survival of these birds.

By monitoring these colonies over time, our model can provide insight into the distribution and status of these colonies. This will help researchers and conservationists understand how the colonies are being impacted by ice loss, temperature changes, and habitat degradation. Integrating diverse datasets into our model's training will enable it to produce crucial information to inform conservation efforts in developing strategies to mitigate the effects of climate change on penguin populations.

IV. REFERENCES

- [1] "Does object size matter in image recognition?," *Augmented Startups*, Online Available: <https://www.augmentedstartups.com/blog/does-object-size-matter-in-image-recognition-unveiling-the-impact-of-size-on-accuracy> [Accessed Mar. 24, 2024]
- [2] "Epochs, batch size, iterations - how they are important," *SabrePC*, Online Available: <https://www.sabrepc.com/blog/Deep-Learning-and-AI/Epochs-Batch-Size-Iterations> [Accessed Mar. 25, 2024]
- [3] A. Sharma, "Leaf disease detection flask app - with source code - 2024," *Machine Learning Projects*, Online Available: <https://machinelearningprojects.net/leaf-disease-detection-flask-app/> [Accessed Apr. 1, 2024].
- [4] A. Sharma, "HealthCure - an all in one medical solution - medical project - 7 disease detections - 2024," *Machine Learning Projects*, Online Available: <https://machinelearningprojects.net/healthcure-medical-project/> [Accessed Mar. 25, 2024].
- [5] "Open images dataset V7 and extensions," *Open Images V7*, Online Available: <https://storage.googleapis.com/openimages/web/index.html> [Accessed Mar. 25, 2024].
- [6] "Rock Dataset Instance Segmentation dataset by Oregon State University," *Roboflow*, Online Available: <https://universe.roboflow.com/oregon-state-university-ny6yy/rock-dataset> [Accessed Mar. 27, 2024].
- [7] "LOCEAN object detection dataset and pre-trained model by LOCEAN," *Roboflow*, Online Available: <https://universe.roboflow.com/locean/locean> [Accessed Mar. 27, 2024].
- [8] L. Jerubbaal, J. Rajkumar, and B. Mahesh, "IMPACT OF IMAGE SIZE ON ACCURACY AND GENERALIZATION OF CONVOLUTIONAL NEURAL NETWORKS.," *Cognizant Technology Solutions Private Limited, ELCOT-S&Z, Sholinganallur, Chennai, India, IJRAR*, vol 6, pp. 70-80, 2019.
- [9] K. Shen, "Effect of batch size on training dynamics," *Medium*, Online Available: <https://medium.com/mini-distill/effect-of-batch-size-on-training-dynamics-21c14f7a716e> [Accessed Apr. 2, 2024].
- [10] "How does epoch affect accuracy in deep learning model?," *GeeksforGeeks*, Online Available: <https://www.geeksforgeeks.org/how-does-epoch-affect-accuracy-in-deep-learning-model/> [Accessed Apr. 3, 2024].
- [11] Scikit-learn Developers, "Precision-recall," *scikit*, Online Available: https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html [Accessed Apr. 5, 2024].
- [12] "Google colaboratory," *Google Colab*, Online Available: <https://colab.research.google.com/github/dules/arch.ipynb> [Accessed Apr. 5, 2024].
- [13] "Nvidia Tesla V100 GPU architecture," The Worlds Most Advanced Data Center GPU, *NVIDIA*, Online Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf> [Accessed Apr. 5, 2024].
- [14] "Who we are," *Woods Hole Oceanographic Institution*, Online Available: <https://www.whoi.edu/who-we-are/> [Accessed Apr. 6, 2024].

- [15] H. Devlin, “Scientists discover emperor penguin colony in Antarctica using satellite images,” *The Guardian*, Online Available: <https://www.theguardian.com/environment/2023/jan/20/scientists-discover-emperor-penguin-colony-in-antarctica-using-satellite-images> [Accessed Apr. 6, 2024].